A Report

On

JDN Bank

A Role-Based Banking Management System

By: -

Nitish Kumar Chaudhary
AP25110010258

Divyanshu Singh
AP25110010036

Kondameeda Jaswanth Kumar
AP25110010589

Prepared in the partial fulfilment of the Project Based Learning of Course

CSE 202 – Object Oriented Programming with C++



SRM University-AP
Amaravati
Andhra Pradesh – 522 240

October 2026

ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to the Vice Chancellor and Dean of SRM University–AP for providing us with the opportunity and academic environment to undertake this project as part of our curriculum.

We extend our heartfelt thanks to our faculty mentor and course instructor for their valuable guidance, encouragement, and support throughout the development of our project, "JDN Bank: A Role-Based Banking Management System." Their suggestions and feedback helped us understand and apply the concepts of Object-Oriented Programming using C++.

We are also grateful to all the individuals who directly or indirectly contributed to the successful completion of this project. Their support and encouragement helped us during the planning, development, and testing phases.

Finally, we would like to thank SRM University–AP for providing us with the necessary resources and facilities to complete this project as part of CSE 202 – Object Oriented Programming with C++.

CERTIFICATE

This is to certify that the project entitled **"JDN Bank: A Role-Based Banking Management System"** has been successfully completed by **Nitish Kumar Chaudhary** (Roll Number: **AP25110010258**), **Divyanshu Singh** (Roll Number: **AP25110010036**), and **Kondameeda Jaswanth Kumar** (Roll Number: **AP25110010589**) as a required academic component of the course **CSE 202: Object Oriented Programming with C++** during Semester 3 of Academic Year 2025–26 in the Department of Computer Science and Engineering at SRM University–AP. This work was executed under the guidance of the undersigned and is deemed to have successfully met all course requirements as of **29th October 2026**.

(Signature of the Course Instructor)

Mr. Rijvan Beg,

Assistant Professor,

Department of Computer Science and Engineering

SRM University, AP

ABSTRACT

JDN Bank is a role-based banking management system developed using C++20 and SQLite as part of the CSE 202 – Object Oriented Programming with C++ course. The project aims to demonstrate the application of object-oriented programming concepts in developing a banking application that manages user accounts and banking operations.

The system provides three roles: Authority, Staff, and User, each with specific permissions and responsibilities. Its features include user registration and login, account management, fund transfers, cheque management, loan processing, EMI payments, and transaction history. The application uses SQLite for persistent data storage and implements approval workflows for selected banking operations. It also incorporates password hashing, input validation, audit logging, and transaction management to support secure and consistent operations.

The project demonstrates the practical use of classes, objects, encapsulation, exception handling, and database integration in C++. It provides an opportunity to understand how software modules work together to manage banking activities. This report presents the system requirements, design, implementation, testing, and outcomes of the JDN Bank application.

Table of Contents

Acknowledgements i

Certificate ii

Abstract iii

INTRODUCTION

1.1 Background of the Project 1

1.2 Problem Statement 2

1.3 Objectives and Scope of the Project 3

1.4 Technologies Used 4

SYSTEM REQUIREMENTS

2.1 Functional Requirements 5

2.2 Non-Functional Requirements 6

2.3 Hardware and Software Requirements 7

SYSTEM DESIGN

3.1 Module and Class Design 8

3.2 Database Design 9

3.3 Fund Transfer and Ledger Design 10

3.4 Loan and Repayment Workflow 11

IMPLEMENTATION

4.1 Development Environment and Build Process 12

4.2 Program Structure and User Interface 13

4.3 Authentication and Role Management 14

4.4 Account and Profile Management 15
4.5 Fund Transfers and Transaction Recording 16
4.6 Cheque and Loan Operations 17
4.7 Database, Validation, and Audit Handling 18

TESTING AND RESULTS

5.1 Software Testing 19
5.2 Test Cases and Results 20
5.3 Application Output 21
5.3.1 Login and Role-Based Menus 21
5.3.2 Account Management and Fund Transfers 22
5.3.3 Cheque and Loan Operations 23
5.3.4 Transaction Records and Database Output 24
5.4 Results and Discussion 25

CONCLUSION 26
REFERENCES 26

Introduction

1.1 Background

Banking management systems play an important role in organising financial operations such as account management, fund transfers, loan processing, and transaction tracking. Computer-based systems help maintain financial information systematically, reduce manual effort, and improve the efficiency of routine operations. The project titled **"JDN Bank: A Role-Based Banking Management System"** is developed using **C++20 and SQLite** as part of the CSE 202 – Object Oriented Programming with C++ course at SRM University–AP. It provides a **terminal-based interface** through which users can access banking services according to their assigned roles.

The system consists of three roles: **Authority, Staff, and User**, each with specific permissions and responsibilities. It supports several operations, including **user registration and login, account management, fund transfers, cheque operations, loan processing, EMI payments, and transaction history**. Certain operations follow an approval workflow in which authorised personnel review requests before completion. This role-based structure organises user responsibilities and demonstrates how access permissions can be implemented in software. The system also allows users to view relevant account and transaction information through its menu options.

The application uses **SQLite as its database** to store user details, account information, transaction records, loan details, and other banking-related data. It incorporates basic security and validation mechanisms, including **password hashing, input validation, restrictions on repeated unsuccessful login attempts, and activity logging**. Database transactions and ledger entries also help maintain consistent records of financial activities. These features demonstrate the importance of data accuracy, controlled access, and proper record management in applications handling financial information.

From an academic perspective, developing JDN Bank provides practical experience in applying **object-oriented programming concepts**, such as **classes, objects, encapsulation, and exception handling**, to a practical problem. The application is divided into modules for the user interface, banking services, database operations, and security functionality.

1.2 Problem Statement

Managing banking operations requires the proper organisation of customer details, account information, financial transactions, and loan records. Without a structured system, maintaining accurate records, tracking financial activities, and handling customer requests can become difficult. Different banking operations also require appropriate validation and record keeping to minimise errors and maintain consistency. In addition, when multiple users are involved in managing banking activities, it is important to define their responsibilities and restrict access to functions according to their roles. The absence of a systematic approach to these tasks can make the management and monitoring of banking operations more complex.

The problem addressed by this project is the need for a **structured banking management system** that can organise common banking activities through a single application. The **JDN Bank system**, developed using **C++20 and SQLite**, aims to address this need through a terminal-based interface that provides different functions to **Authority, Staff, and User** roles. The system supports account management, fund transfers, cheque operations, loan processing, EMI payments, and transaction history. However, these operations require appropriate handling of user permissions, transaction details, and requests that may need approval from authorised personnel. Therefore, the application incorporates role-based access and approval workflows to organise the processing of selected banking activities.

Another challenge is ensuring that financial operations are validated and their records are maintained consistently. Incorrect inputs, repeated unsuccessful login attempts, and improperly recorded transactions can affect the reliability of an application. To address these concerns at an academic project level, JDN Bank incorporates **input validation, password hashing, database transactions, ledger entries, and activity logging**. These mechanisms demonstrate basic approaches to handling user access, recording financial activities, and maintaining organised data. Thus, the project focuses on addressing the challenges of banking record management, controlled access, and transaction processing by integrating **object-oriented programming and database management** into a single application.

1.3 Objectives and Scope

The main objective of JDN Bank is to develop a **role-based banking management system using C++20 and SQLite** that demonstrates the practical application of object-oriented programming concepts. The project aims to organise banking operations, maintain relevant records, and provide appropriate functionality according to user roles.

The specific objectives of the project are:

1. To apply **classes, objects, encapsulation, and exception handling** in a practical C++ application.
2. To implement **role-based access** for Authority, Staff, and User.
3. To support account management, fund transfers, cheque operations, loan processing, EMI payments, and transaction history.
4. To integrate **SQLite database management** for persistent storage and retrieval of application data.
5. To demonstrate basic security, input validation, approval workflows, and transaction consistency.

The scope of the project includes the implementation of these banking-related features through a terminal-based interface. It also covers user and account management, selected request approvals, transaction recording, and loan-related operations. The application is intended for **academic demonstration** and does not connect to real banking networks or process actual interbank payments.

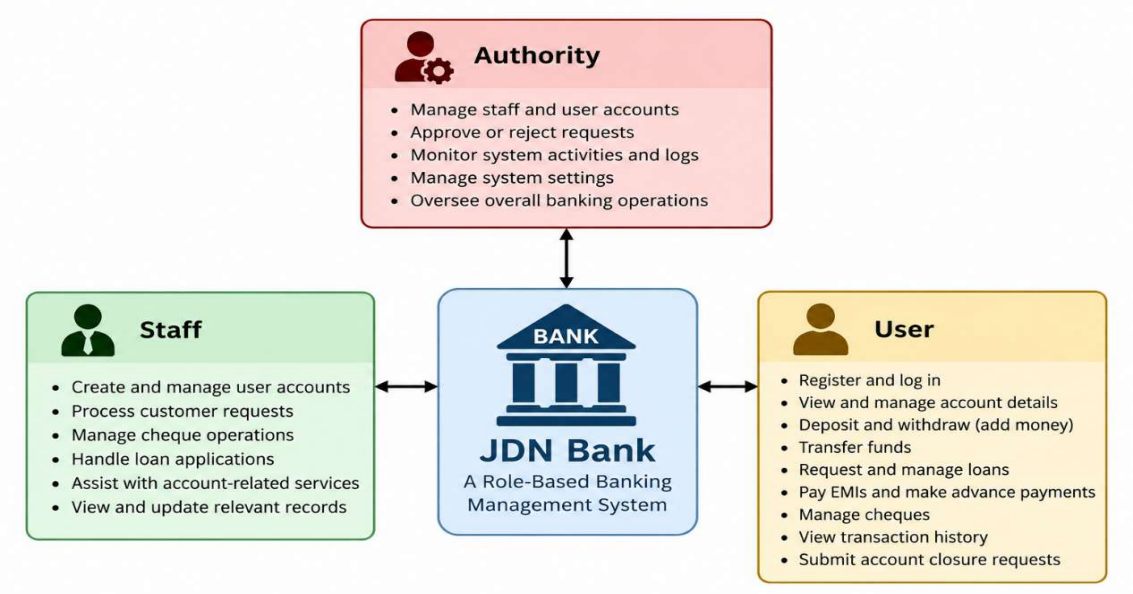


Figure 1.1: Role-Based Structure of JDN Bank

1.4 Technologies Used

JDN Bank uses a combination of programming and database technologies to implement its features and manage application data. The primary technologies used in the project are **C++20 and SQLite**.

- **C++20:** The main programming language used to implement banking operations, role-based functionality, input validation, and error handling. Its object-oriented features help organise the application into classes and modules.
- **SQLite:** A relational database used to store and retrieve user details, account information, transaction records, loan information, and other application data.
- **SQL (Structured Query Language):** Used to insert, retrieve, update, and manage records in the SQLite database.
- **Terminal-Based Interface:** Provides a menu-driven way for users to navigate the application and access available functions.
- **Modular Program Structure:** Separates components such as the user interface, banking services, database operations, security, and shared data types into different source and header files.

These technologies work together to provide **banking functionality, persistent data storage, and organised program structure**. Their integration demonstrates how a C++ application can interact with a relational database to implement a complete academic software project.

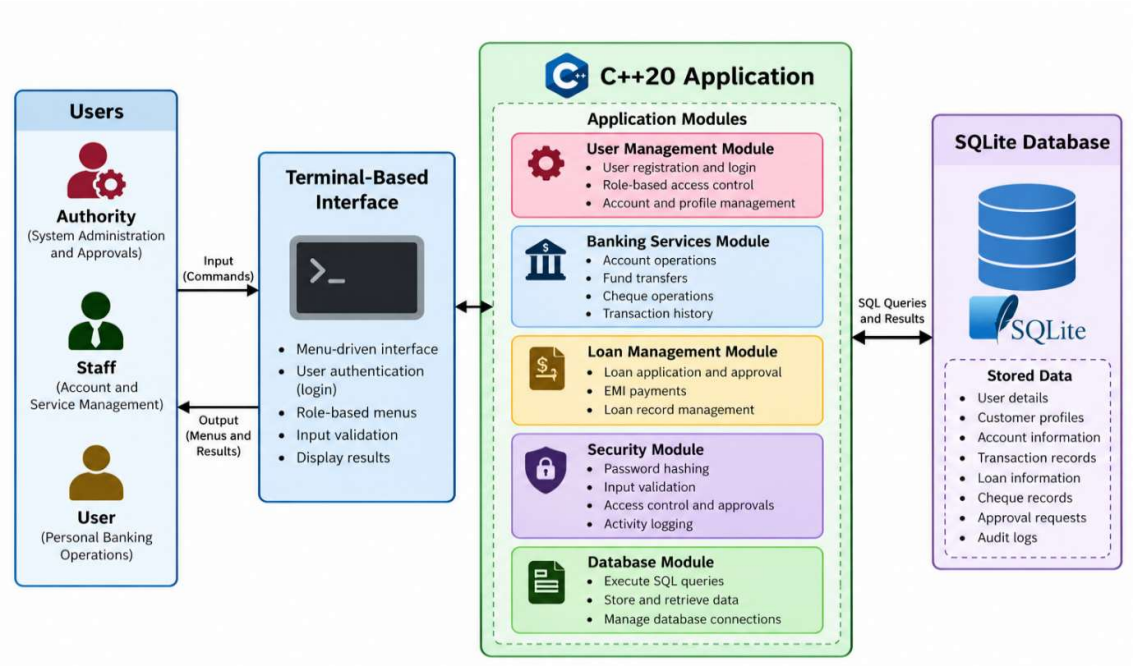


Figure 1.2: Technology Architecture of JDN Bank

System Requirements and Analysis

2.1 Functional Requirements

Functional requirements describe the operations a software system must provide. In **JDN Bank**, these requirements focus on authentication, account management, banking transactions, loan processing, and role-based activities. The system provides different functions for **Authority, Staff, and User**.

The main functional requirements are:

1. **User Registration and Login:** Allows users to register and log in using their credentials, with password hashing and restrictions on repeated failed login attempts.
2. **Role-Based Access:** Provides different functions and permissions for Authority, Staff, and User.
3. **Account and Profile Management:** Supports customer profile updates and account-related operations, including closure requests.
4. **Fund Transfers:** Supports **IMPS, NEFT, RTGS**, and cheque-related transfers according to the system's rules.
5. **Transaction History:** Records banking activities and allows users to view relevant transaction details.
6. **Cheque Management:** Supports cheque-book requests and related cheque operations.
7. **Loan Management:** Allows users to request loans and authorised personnel to review, approve, and disburse them.
8. **Loan Repayment:** Supports EMI payments and advance repayments, with payment records stored in the database.
9. **Request Approval:** Allows authorised personnel to process selected requests, including loan, add-money, and account-related requests.
10. **Database and Activity Records:** Uses **SQLite** to store user, account, transaction, loan, cheque, and approval records, along with relevant activity logs.

These requirements define the core features of JDN Bank and guide its design and testing.

2.2 Non-Functional Requirements

Non-functional requirements describe the quality, reliability, and performance expected from a software system. In **JDN Bank**, these requirements focus on maintaining secure access, consistent data, ease of use, and an organised program structure. They help ensure that the application performs its intended functions reliably within the scope of the project.

The main non-functional requirements of the application are listed below:

1. **Security:** The system uses password hashing and role-based access to help protect user accounts and restrict access to authorised functions. It also limits repeated unsuccessful login attempts.
2. **Reliability:** Database transactions help maintain consistency during important operations, such as fund transfers, reducing the risk of incomplete updates to related records.
3. **Usability:** The application provides a **terminal-based, menu-driven interface** that allows users to navigate banking services and select functions according to their roles.
4. **Data Integrity:** The system maintains transaction records and ledger entries to keep financial activities organised and support consistent account records.
5. **Maintainability:** The program is divided into modules for the user interface, banking services, database operations, security, and shared data types. This separation makes the code easier to understand and modify.
6. **Input Validation:** The application checks user inputs and handles invalid entries to reduce errors during registration, account management, and banking operations.
7. **Data Persistence: SQLite** stores user details, account information, transactions, and loan records. This allows relevant information to remain available when the application is closed and reopened.

These requirements contribute to the **security, reliability, usability, and maintainability** of JDN Bank and support its use as an academic demonstration of a banking management system.

In addition, the system is designed to organise banking operations through clearly defined modules and controlled workflows. These qualities make the application easier to test, maintain, and improve while helping ensure that user requests and financial records are handled in a structured manner.

2.3 Hardware and Software Requirements

JDN Bank requires suitable hardware and software to compile, run, and manage the application. Since the project is developed using **C++20** and **SQLite**, the system must support a compatible C++ compiler and the required database library.

Hardware Requirements

The basic hardware requirements for running the application are:

1. **Processor:** A computer with a modern processor capable of running a C++ application.
2. **RAM:** Sufficient memory to run the application and database operations smoothly.
3. **Storage:** Available disk space for the source code, compiled program, and SQLite database file.
4. **Input and Output Devices:** A keyboard and display for interacting with the terminal-based interface.

Software Requirements

The software required for developing and running JDN Bank includes:

1. **Programming Language:** C++20 is used to implement the application's logic and banking operations.
2. **Compiler:** A C++ compiler that supports the C++20 standard is required to compile the source code.
3. **Database:** SQLite is used for storing and retrieving application data.
4. **Development Environment:** A suitable code editor or IDE is required to write, organise, and maintain the source files.
5. **Operating System:** An environment that supports the required C++ compiler and SQLite library is needed to build and run the application.

These requirements provide the basic environment for developing, executing, and testing JDN Bank. The application is designed as a **terminal-based academic project** and does not require specialised banking hardware or infrastructure.

System Design

3.1 Module and Class

JDN Bank follows a **modular program structure** in which different components handle specific responsibilities. This organisation makes the application easier to understand, maintain, and extend. The major components include the user interface, banking services, database management, security, and shared data types.

The **user interface module** manages the terminal menus and user interaction. The **banking service module** handles operations such as account management, fund transfers, and loan processing. The **database module** manages communication with SQLite, while the **security module** supports password hashing and authentication-related functions. Shared data types help represent common information used across the application.

The system provides three roles: **Authority, Staff, and User**. Each role has access to functions according to its responsibilities. By separating user interaction, banking logic, database operations, and security functions, JDN Bank demonstrates the use of modular design in a C++ application.

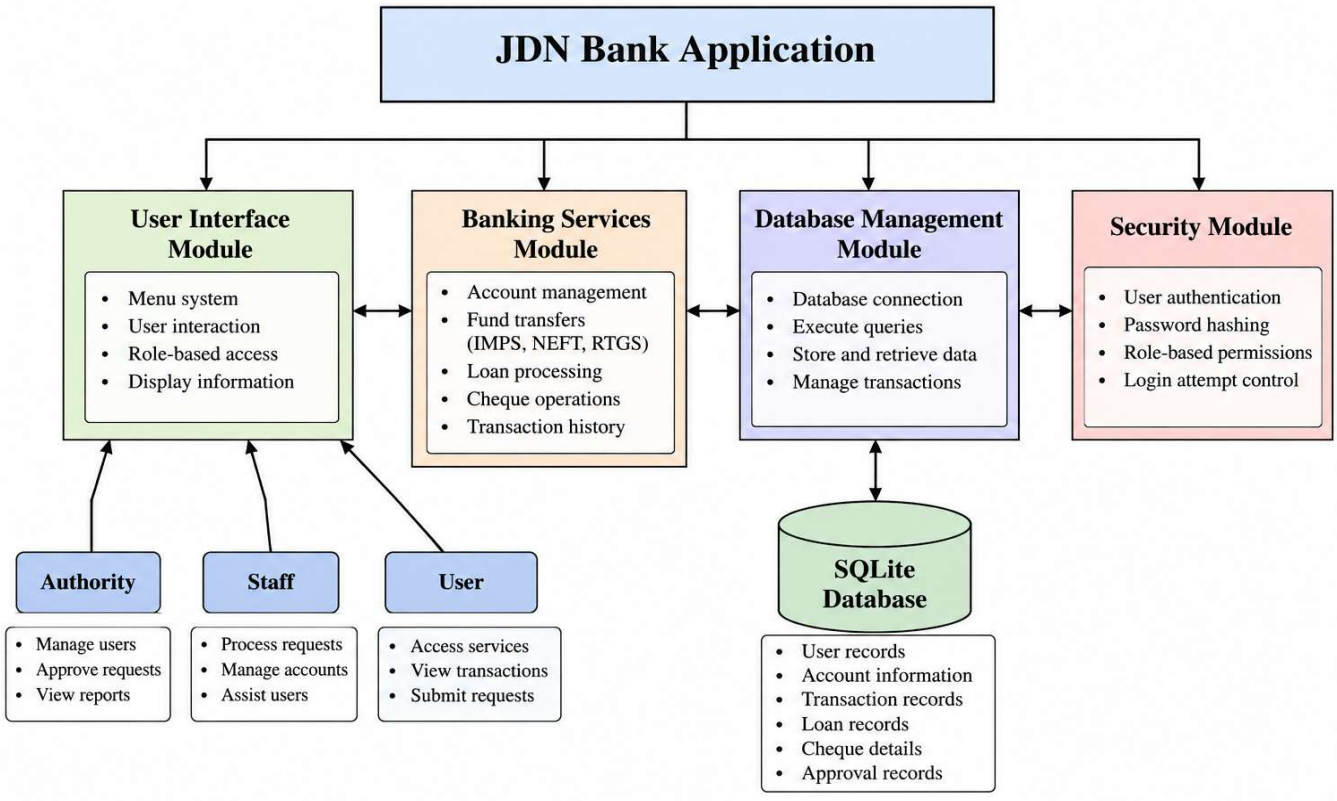


Figure 3.1: Module Structure of JDN Bank

3.2 Database

JDN Bank uses **SQLite** as its relational database to store and manage information required for banking operations. The database maintains records related to users, customer profiles, accounts, transactions, loans, cheque operations, and approval requests. This allows the application to retrieve and update information whenever required.

The database is organised into multiple tables, with each table handling a specific type of information. The **users** table stores user account details, while **customer_profiles** and **accounts** maintain customer and account-related information. The **bank_transactions** and **ledger_entries** tables record transaction details and corresponding financial entries. Other tables, such as **loans**, **loan_payments**, **cheques**, and **approval_requests**, support loan management, repayments, cheque operations, and request processing. The **audit_logs** and **fraud_flags** tables maintain records related to system activities and transaction risk indicators.

This database structure separates different categories of information and supports the application's banking workflows. By using SQLite, JDN Bank provides **persistent data storage** and organised access to records throughout its operations.

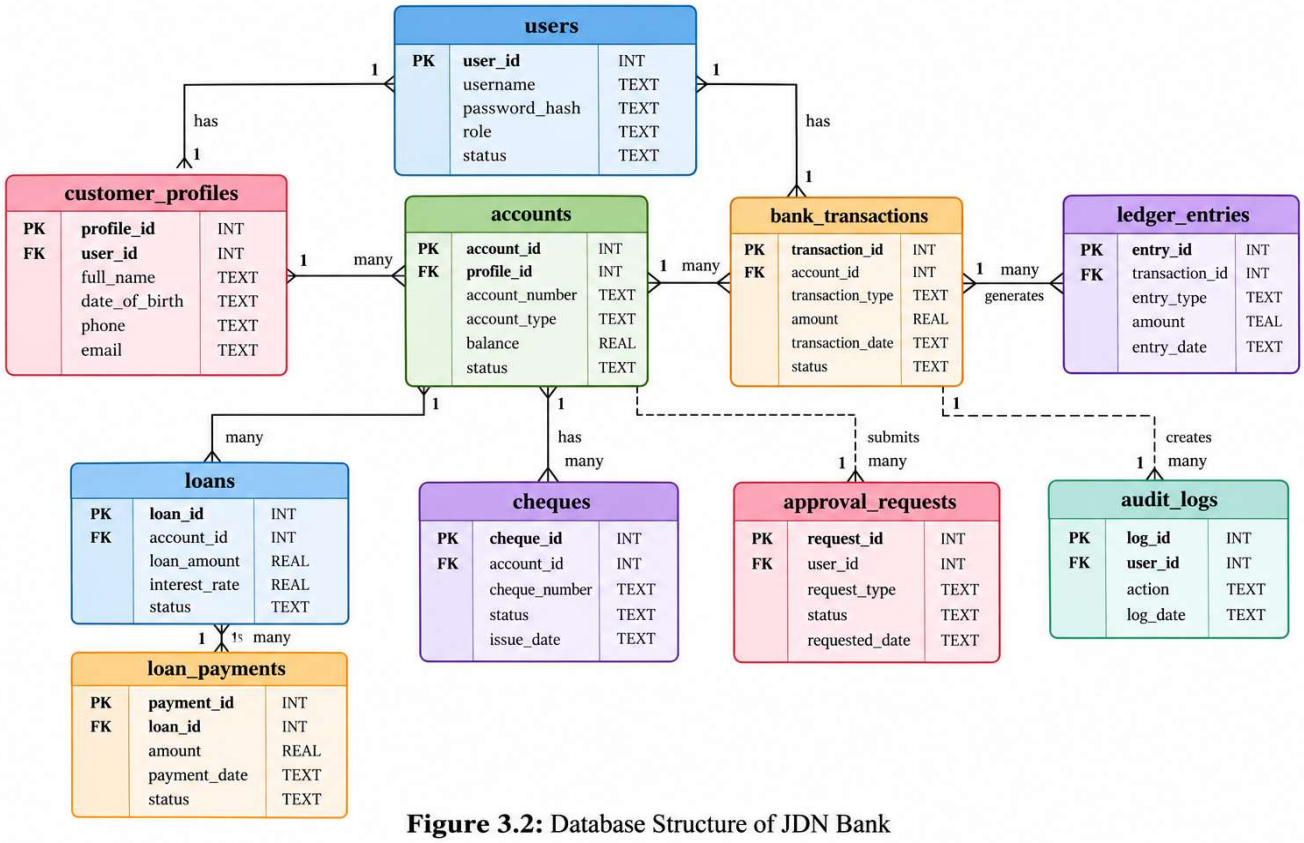


Figure 3.2: Database Structure of JDN Bank

3.3 Fund Transfer and Ledger

JDN Bank implements fund transfers through a structured process that updates account balances and records transaction details. The system supports transfer methods such as **IMPS, NEFT, and RTGS**, along with cheque-related operations, according to the implemented rules.

During a transfer, the application validates the required details and checks whether the transaction can be processed. When the transfer is valid, the relevant account balances are updated and transaction information is stored in the SQLite database. The system uses database transactions to help ensure that related changes are completed consistently or rolled back if an operation fails.

The application also maintains **ledger entries** to record the debit and credit sides of financial transactions. These entries provide a record of how funds move between accounts and support transaction tracking. The use of transaction records and ledger entries demonstrates the importance of **data consistency and financial record keeping** in a banking management system.

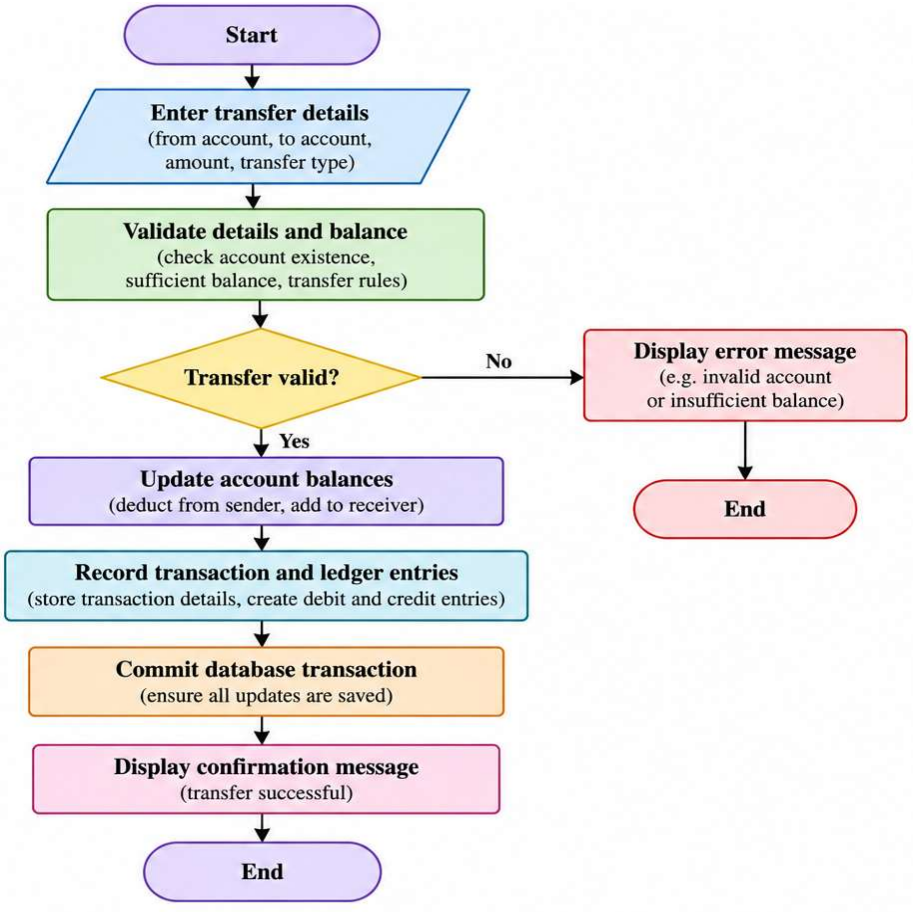


Figure 3.3: Fund Transfer Flowchart

3.4 Loan and Repayment Workflow

JDN Bank includes a loan management module that allows users to submit loan requests and supports their review and processing by authorised personnel. The system maintains loan-related information, including the requested amount, interest rate, repayment period, and loan status, in the SQLite database.

The loan process begins when a user submits a loan request. The request is then reviewed through the application's approval workflow. Once approved, the loan can proceed to disbursal according to the system's implemented operations. The application maintains the relevant loan records to track the status of each request.

The system also supports **EMI payments and advance repayments**. Payment details are stored in the database, allowing repayment activities to be recorded and tracked. The **loans** and **loan_payments** tables support the organisation of loan details and repayment records.

This workflow demonstrates how loan applications, approval processes, disbursal, and repayment records can be managed within a single banking application.

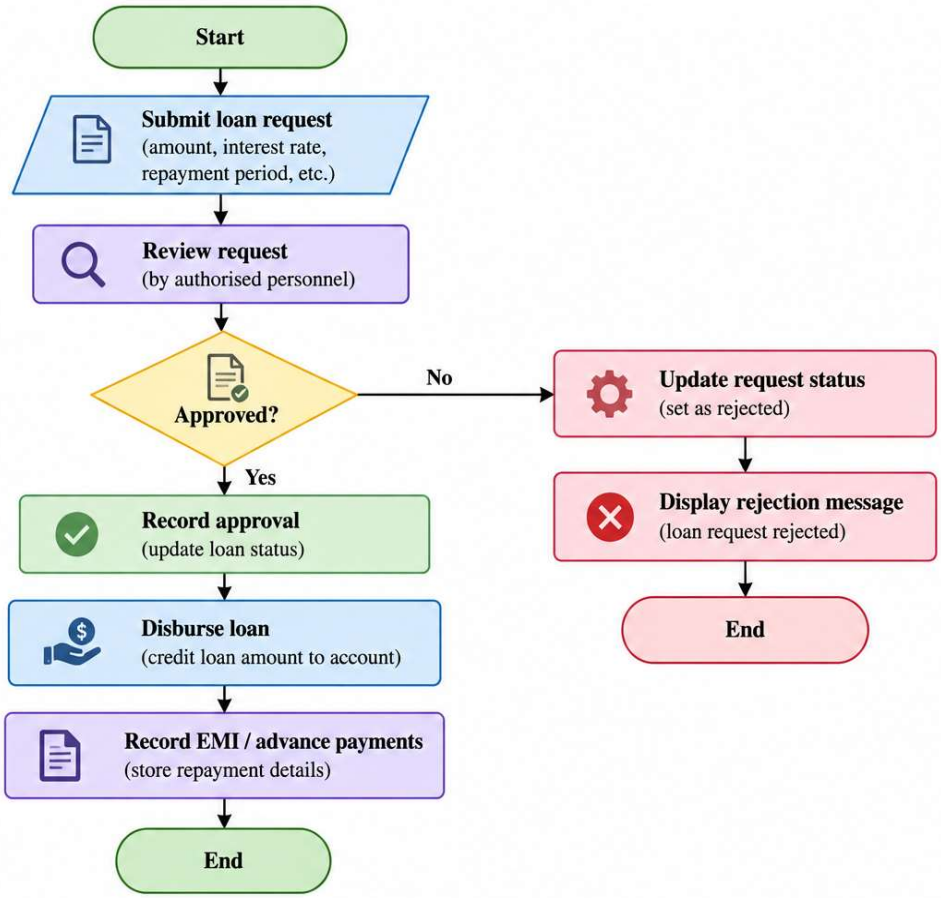


Figure 3.4: Loan Processing Flowchart

IMPLEMENTATION

4.1 Development Environment and Build Process

JDN Bank is implemented in **C++20**, with **SQLite** used to store and manage application data. The project is organised into source files (.cpp) and header files (.hpp), separating the user interface, banking operations, database functionality, security, and shared data types. This structure allows the application to be compiled as a single program while keeping its implementation organised across multiple files.

The main source files include main.cpp, Ui.cpp, BankService.cpp, Database.cpp, Security.cpp, and Types.cpp, together with their corresponding header files. The application entry point is defined in main.cpp, while the remaining modules provide the functionality required to run the banking operations. SQLite integration enables the program to retain relevant records beyond a single execution.

To build the application, a **C++ compiler supporting the C++20 standard** and the required SQLite development library must be available in the development environment. The source files need to be compiled and linked with the appropriate SQLite library so that database operations can be performed at runtime. The resulting executable can then be launched from a terminal.

During execution, the application displays its menu-driven interface and interacts with the SQLite database as users perform supported operations. Database records are used to maintain information such as user details, accounts, transactions, loan records, and requests. This connection between the compiled C++ application and persistent storage forms the basis of the system's implementation.

The build and execution process therefore brings together the project's **C++20 source code, modular file structure, and SQLite dependency**. It provides the foundation for running and testing the banking functions described in the following sections.

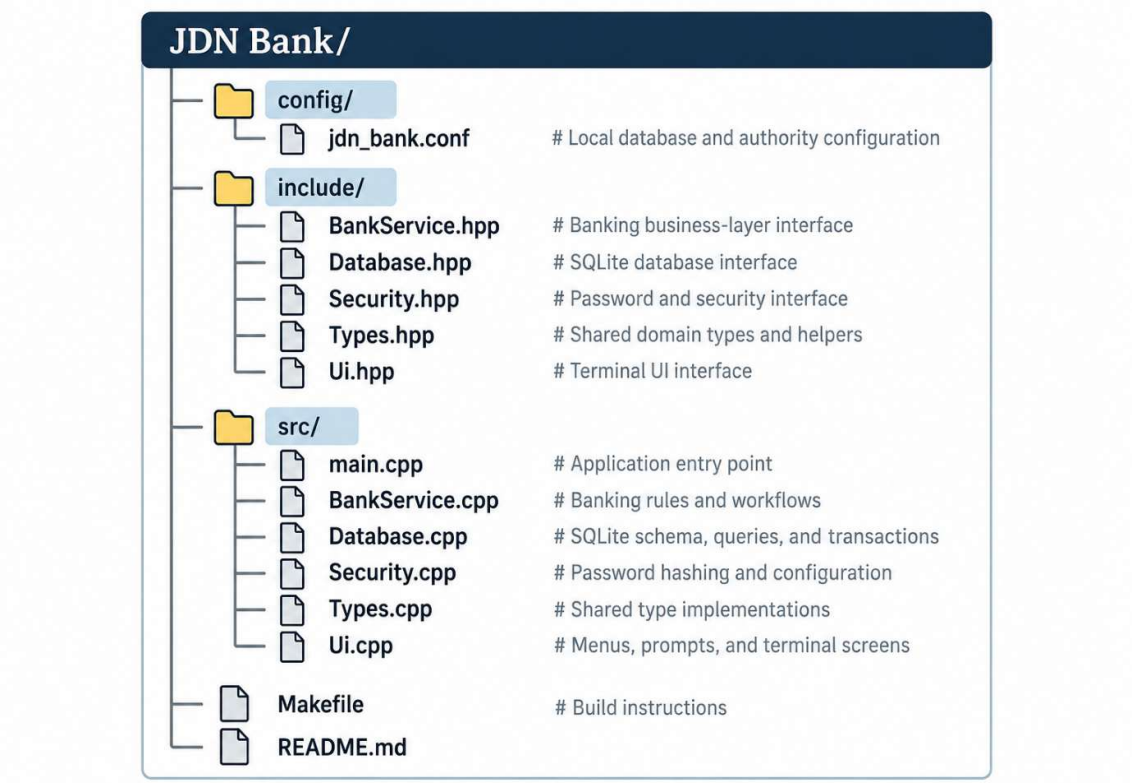
4.2 Program Structure and User Interface

JDN Bank is organised into source and header files that separate program execution from user interaction. The **main.cpp** file serves as the entry point, initiating the application, while **Ui.hpp and Ui.cpp** define and implement the user interface.

The application uses a **terminal-based, menu-driven interface**. Users select available options, enter the required information, and receive responses through the terminal. The menus provide access to features such as account management, fund transfers, cheque services, and loan operations.

The interface connects user input with the relevant banking functionality. Available options depend on the user's role—**Authority, Staff, or User**—and the permissions associated with it. This allows users to access the functions intended for their responsibilities without directly interacting with the database.

Separating interface code from the main banking logic improves **code organisation, readability, and maintainability**. It also provides a straightforward way to interact with and test the application's features.



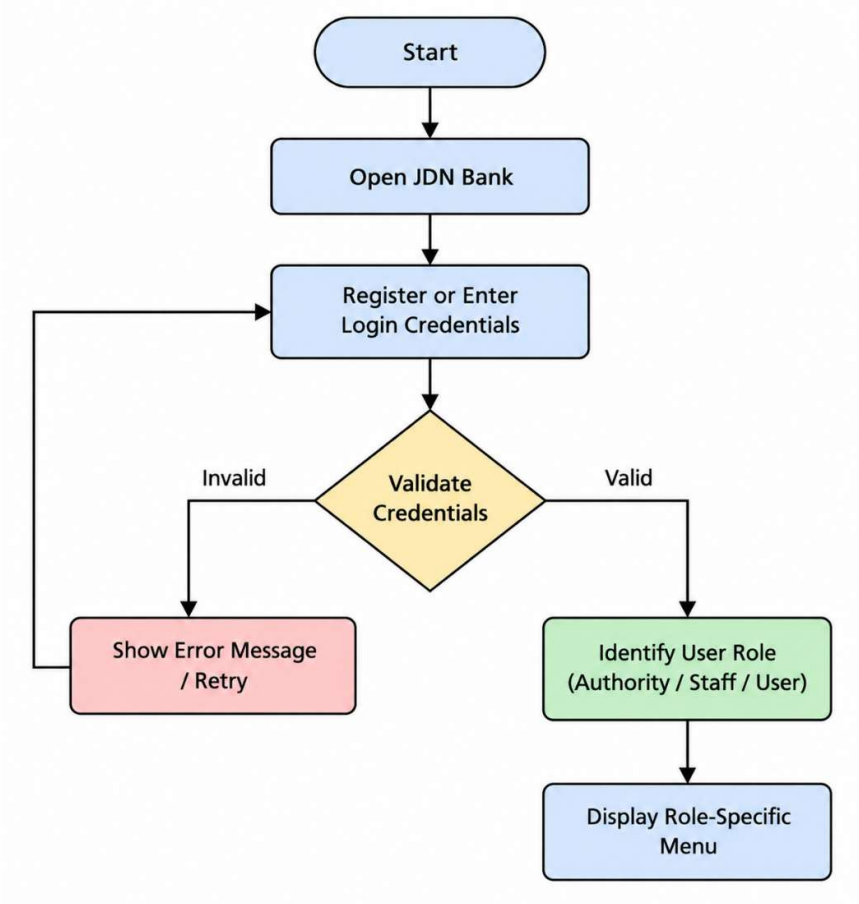
4.3 Authentication and Role Management

JDN Bank implements **user registration and login** to control access to the application. During registration, the system collects the required user information and stores account details in the SQLite database. Passwords are processed using a **password-hashing mechanism** rather than being stored as plain-text passwords.

During login, the application verifies the supplied credentials and applies restrictions to repeated unsuccessful attempts. After successful authentication, the system identifies the user's role and provides access to the corresponding functions.

The application supports three roles: **Authority, Staff, and User**. Each role has different responsibilities and access permissions for banking operations, such as managing requests, handling accounts, and using customer services. This role-based approach helps restrict access to functions according to the user's assigned responsibilities.

Overall, the authentication and role-management modules provide a structured way to manage user access within JDN Bank.



4.4 Account and Profile Management

JDN Bank provides features for managing customer profiles and bank accounts. Customer information is maintained in the **customer_profiles** table, while account-related details are stored in the **accounts** table. These records allow the application to retrieve and update information required for banking operations.

Users can access supported profile and account-related functions through the terminal interface. The system also supports **account closure requests**, which can be submitted for processing through the application's request-handling workflow.

The account management functionality connects customer information with banking services and helps maintain organised records in the SQLite database. By separating profile and account data, the application keeps customer details and account information structured for further operations.

4.5 Fund Transfers and Transaction Recording

JDN Bank implements fund transfers through the banking service module, allowing users to transfer money using supported methods such as **IMPS, NEFT, and RTGS**. The application collects the required transfer details, validates the input, and checks the conditions necessary to process the transaction.

When a transfer is accepted, the application updates the relevant account balances and records the transaction in the SQLite database. The **bank_transactions** table stores transaction information, while the **ledger_entries** table maintains the corresponding debit and credit entries. These records help track the movement of funds between accounts.

The application uses **database transactions** to handle related updates as a single operation. If an operation fails, the transaction can be rolled back to help prevent incomplete changes to account balances and financial records. This is important because a transfer involves multiple related updates that should remain consistent.

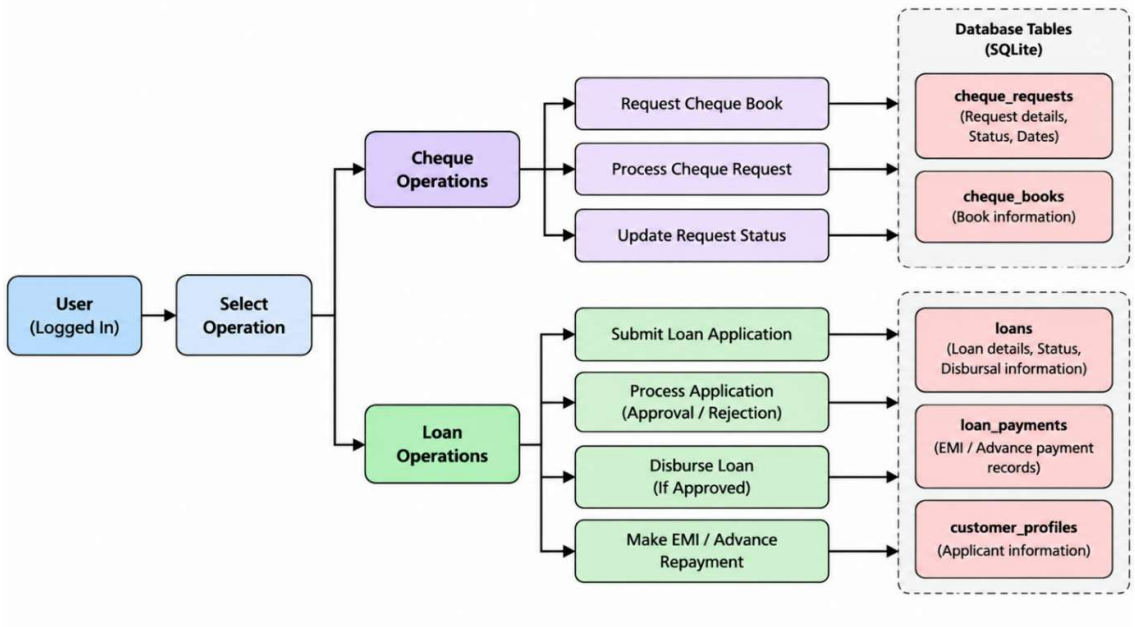
4.6 Cheque and Loan Operations

JDN Bank provides cheque-related and loan-related features to support additional banking activities. The cheque management functionality allows users to submit **cheque-book requests** and supports cheque operations through the application's banking service module. Relevant cheque information is stored in the SQLite database, allowing the system to maintain records associated with cheque activities.

The loan management module allows users to submit **loan applications** for review through the application's approval workflow. Authorised personnel can process these requests, and approved loans can proceed to disbursal according to the implemented functionality. Loan details and their status are maintained in the database to support loan processing and record keeping.

The application also supports **EMI payments and advance repayments**. When a repayment is processed, the relevant payment information is recorded in the database. The loans and loan_payments tables organise loan details and repayment records, helping the application track loan-related activities.

These operations connect request handling, approval processes, and database storage. By combining cheque services with loan processing and repayment features, JDN Bank demonstrates how multiple banking functions can be implemented within a single C++ application.



4.7 Database, Validation, and Audit Handling

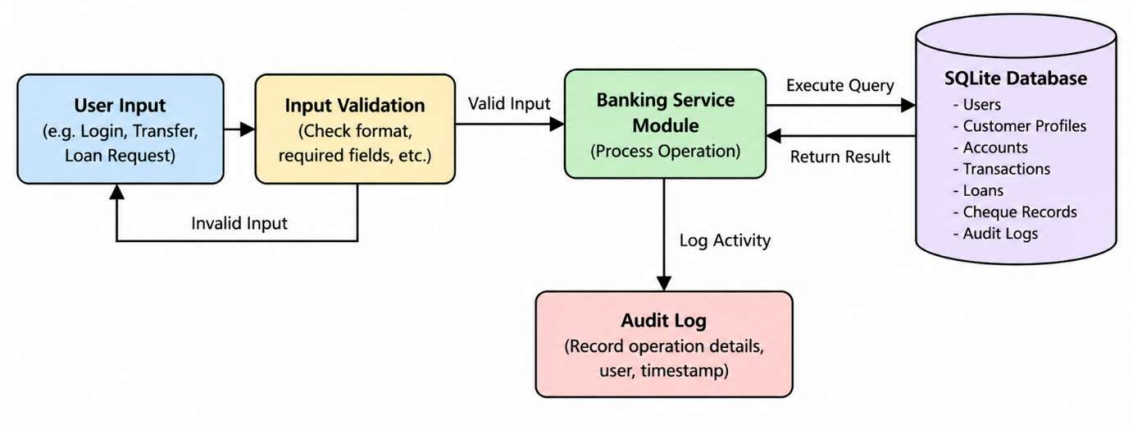
JDN Bank uses **SQLite** to store and manage information required for its banking operations. The Database module handles database connectivity and query execution, while the banking service module uses these operations to retrieve and update records. The database stores information related to users, customer profiles, accounts, transactions, loans, cheque operations, and approval requests.

Input validation is performed to check information entered by users before processing supported operations. This helps identify invalid inputs and reduces errors during registration, account management, fund transfers, and other banking activities. The application also uses error-handling mechanisms to respond to unsuccessful operations and prevent invalid requests from being processed normally.

For important financial operations, **database transactions** help maintain consistency between related updates. If an operation fails, rollback handling can reverse uncommitted changes. This is particularly relevant to transfers, where account balances and transaction records must be updated together.

The system also maintains **audit logs** to record relevant application activities. These records support tracking of operations performed within the system. In addition, password hashing and restrictions on repeated failed login attempts provide basic safeguards for account access.

Together, database management, input validation, transaction handling, and audit logging help JDN Bank maintain organised records and handle banking operations more consistently. These mechanisms demonstrate important software development practices, although the application remains an **academic project rather than a production banking system**.



TESTING AND RESULTS

5.1 Software Testing

Software testing is performed to verify that JDN Bank's features work as expected and that the application handles user inputs and banking operations correctly. Testing focuses on registration and login, role-based access, account management, fund transfers, cheque operations, loan processing, and repayment functions.

The application should be tested with valid and invalid inputs to check whether operations are processed correctly and errors are handled appropriately. Database records should also be examined to confirm that successful operations are stored and that failed financial operations do not leave incomplete updates.

Testing helps identify implementation errors and verify the interaction between the C++ modules and the SQLite database. The results should be documented based on the actual execution of the application.

5.2 Test Cases and Results

Test cases are used to verify the main functions of JDN Bank and check whether the application produces the expected results.

TEST CASE	EXPECTED RESULT
REGISTRATION AND LOGIN	Valid credentials are accepted.
ROLE-BASED ACCESS	Role-specific menus are displayed.
FUND TRANSFER	Balances and transaction records are updated.
INVALID TRANSFER	Invalid requests are rejected.
CHEQUE OPERATIONS	Requests are recorded correctly.
LOAN PROCESSING	Loan requests and repayments are recorded.
DATABASE PERSISTENCE	Records remain available after restarting.

These test cases cover the major functions of JDN Bank. The actual results should be verified through application testing.

5.3 Application Output

JDN Bank provides a terminal-based interface for performing banking operations. The application displays role-specific menus, processes user requests, and presents the results of operations. The following subsections demonstrate the major features through application outputs.

5.3.1 Login and Role-Based Menus

The application allows users to log in and access functions according to their assigned roles: Authority, Staff, or User. Each role provides access to the relevant operations and menu options.



Figure 5.1 Landing Page



Figure 5.2 Role-Based Registration

5.3.2 Account Management and Fund Transfers

The account management and fund transfer features allow users to access account-related functions and initiate supported transfers. The application processes valid requests and records the relevant transaction details.



Figure 5.3 Account Management



Figure 5.4 Account Transfer using IMPS

5.3.3 Cheque and Loan Operations

The application supports cheque-book requests, loan applications, approval workflows, and repayment operations. These functions help demonstrate the implementation of additional banking services.

```
+--------------------------------------------------------------------+
|                            JDN BANK                                |
+--------------------------------------------------------------------+
  Applications

  Application
    [1] Cheque book
    [2] Loan
    [3] Add money
    [4] Passbook
    [5] Back
  Choose: 1
  Delivery address: D.L.F
  Leaves (1-100): 10
Cheque book request sent to Staff for approval.

  Press Enter to continue...
```

Figure 5.5 Cheque Book Request

```
+--------------------------------------------------------------------+
|                            JDN BANK                                |
+--------------------------------------------------------------------+
  Cheque requests

  request_id  name        metadata              status    created_at
  ----------  ----------  --------------------  --------  -------------------
  3           Riya Patel  300000000038|10|D.L.F  APPROVED  2026-10-29 00:44:31
  cheque_number  status    account_number  name        created_at
  -------------  --------  --------------  ----------  -------------------
  CHQ771072342   ISSUED    300000000038    Riya Patel  2026-10-29 00:44:46
  CHQ7387103e8   ISSUED    300000000038    Riya Patel  2026-10-29 00:44:46
  CHQ7a49a7a49   ISSUED    300000000038    Riya Patel  2026-10-29 00:44:46
  CHQ7a471de43   ISSUED    300000000038    Riya Patel  2026-10-29 00:44:46
  CHQ73fdc1cbf   ISSUED    300000000038    Riya Patel  2026-10-29 00:44:46
  CHQ71638eba7   ISSUED    300000000038    Riya Patel  2026-10-29 00:44:46
  CHQ715c6bf35   ISSUED    300000000038    Riya Patel  2026-10-29 00:44:46
  CHQ7d80f4509   ISSUED    300000000038    Riya Patel  2026-10-29 00:44:46
  CHQ70bee2fa7   ISSUED    300000000038    Riya Patel  2026-10-29 00:44:46
  CHQ7d831955a   ISSUED    300000000038    Riya Patel  2026-10-29 00:44:46
```

Figure 5.6 Cheque Leaves

5.3.4 Transaction Records and Database Output

JDN Bank stores relevant banking information in SQLite, including transaction, account, and loan records. These records can be examined to verify that processed operations are retained in the database.

```
visible_id  name         account_number  login
----------  -----------  --------------  --------
1           Riya Patel   300000000038    user1
2           Kabir Singh  300000000046    user2
User s.no, or 0: 1
name                : Riya Patel
phone               : 9876500003
dob                 : 1995-09-21
email               : riya@example.com
address             : JDN Bank Customer Address
pan                 : CDEFG3456H
aadhaar             : 345634563456
login               : user1
account_number      : 300000000038
credit_score        : 650
balance             : ₹900.19
created_at           name        account_number  description
-------------------  ----------  --------------  ---------------------------------
2026-10-29 00:45:39  Riya Patel  300000000038    Loan #2; 20 monthly instalments.
2026-10-29 00:45:18  Riya Patel  300000000038    300000000038 leaves=10
2026-10-29 00:44:54  Riya Patel  300000000038    Authenticated successfully.
2026-10-29 00:44:31  Riya Patel  300000000038    300000000038 leaves=10
2026-10-29 00:43:57  Riya Patel  300000000038    JDN2026-10-29-102f3577 IMPS ₹100.00
2026-10-29 00:43:49  Riya Patel  300000000038    Authenticated successfully.
2026-10-29 00:41:31  Riya Patel  300000000038    Authenticated successfully.
2026-10-26 10:57:53  Riya Patel  300000000038    Authenticated successfully.
2026-10-26 10:57:25  Riya Patel  300000000038    Authenticated successfully.
2026-10-26 10:56:40  Riya Patel  300000000038    Authenticated successfully.
2026-10-26 10:53:56  Riya Patel  300000000038    Loan #1; 6 monthly instalments.
```

Figure 5.7 Transaction Info

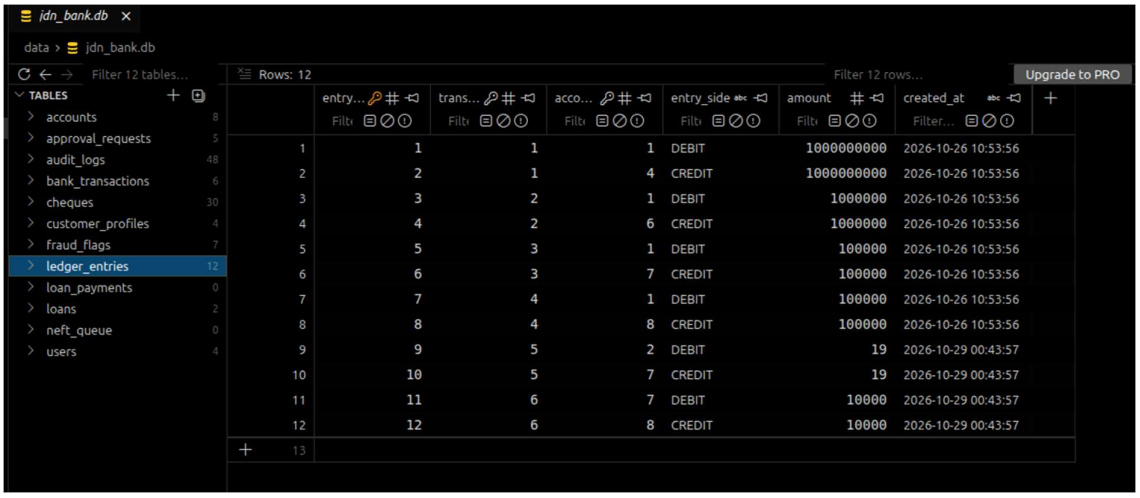


Figure 5.8 Database Records

5.4 Results and Discussion

JDN Bank integrates multiple banking functions into a single **C++20 application** supported by an SQLite database. Its modular structure separates user interaction, banking operations, database management, and security-related functionality.

The implemented features include **user authentication, role-based access, account management, fund transfers, cheque operations, loan processing, and repayment recording**. The application also maintains transaction and ledger records to support the tracking of banking activities.

The project demonstrates the practical application of **object-oriented programming, database integration, input validation, and transaction handling**. Actual testing results should be used to confirm the behaviour of individual features and identify any remaining limitations.

Overall, JDN Bank fulfils its purpose as an academic demonstration of a role-based banking management system. It also provides a foundation for future improvements in usability, testing, and security.



Figure 5.9 Exit Page

CONCLUSION

JDN Bank is a **role-based banking management system** developed using C++20 and SQLite as part of the CSE 202 – Object Oriented Programming with C++ course. The project integrates essential banking functions, including user authentication, account management, fund transfers, cheque operations, loan processing, and repayment recording.

The application demonstrates the practical use of **object-oriented programming, modular design, database integration, and transaction handling**. Its role-based structure provides different functions for Authority, Staff, and User, while SQLite supports the persistent storage of banking records.

Through this project, the importance of structured software design, input validation, data consistency, and organised record keeping can be understood in a practical context. Overall, JDN Bank demonstrates how C++ and database technologies can be combined to develop a banking management application for academic purposes.

REFERENCES

- Stroustrup, B. *The C++ Programming Language*. 4th ed., Addison-Wesley Professional, 2013.
- ISO C++. *C++ Documentation and Resources*. https://isocpp.org/
- SQLite. *SQLite Documentation*. https://www.sqlite.org/docs.html
- SQLite. *SQLite C/C++ Interface*. https://www.sqlite.org/cintro.html
- SRM University–AP. *CSE 202 – Object Oriented Programming with C++: Course Materials and Project Guidelines*.